

AUTOMATISATION D'UNE INFRASTRUCTURE LINUX AVEC BASH

*Documentation du script `techcorp_setup.sh` : de la commande manuelle au
script idempotent*

Rachidi Tchêgoun AGRO

Élève-ingénieur - ENSGMM / UNSTIM, Abomey, Bénin

Août 2026

Objectif de ce document

Ce document explique le script Bash construit pour automatiser la mise en place d'une infrastructure de fichiers multi-services (TechCorp), qui avait auparavant été réalisée entièrement à la main. Il couvre le raisonnement derrière chaque fonction, les erreurs rencontrées pendant la construction, et les preuves d'exécution réelles.

Aucun prérequis en programmation n'est nécessaire pour suivre ce document : chaque notion est introduite avec son utilité concrète avant d'être utilisée.

1 Pourquoi automatiser

La mise en place manuelle des 4 services (RH, IT, Finance, Direction) demandait de retaper, service par service, la même séquence de 7 commandes (création du groupe, création des utilisateurs, dossier, permissions, SGID). Cette répétition manuelle est une source directe d'erreurs — plusieurs fautes de frappe réelles sont survenues lors de la phase manuelle du projet (espace mal placé dans `chown`, ordre des arguments inversé dans `chmod`).

Un script transforme cette séquence en une procédure écrite une seule fois, réutilisable à volonté, sans dépendre de la vigilance humaine à chaque exécution.

2 Les bases du script

Le shebang

Tout script commence par une ligne spéciale indiquant quel interpréteur doit l'exécuter :

```
#!/bin/bash
```

Deux façons de lancer un script

Méthode	Effet
bash script.sh	Bash lit et exécute le fichier ; le shebang et le droit d'exécution ne sont pas nécessaires
./script.sh	Le système exécute le fichier comme un programme ; shebang et droit d'exécution (x) obligatoires

Un fichier `.sh` fraîchement créé n'a jamais le droit d'exécution par défaut (umask) — `chmod +x` (ou 700 pour un usage strictement personnel) est nécessaire avant tout lancement via `./`.

3 Variables, substitution de commande, entrées utilisateur

```
nom="RH"
echo "Service : $nom"

resultat=$(whoami)  # exécute whoami, capture son résultat dans la variable

read service        # met le script en pause, attend une saisie clavier
```

Règle systématiquement appliquée dans le script final : toute variable est utilisée entre guillemets ("`$variable`") pour éviter des comportements imprévisibles si la valeur contient un espace ou est vide.

4 Tester avant d'agir : conditions et codes de sortie

Chaque commande Linux renvoie un code de sortie : 0 en cas de succès, un autre nombre en cas d'échec, accessible via la variable spéciale `$?` . C'est ce mécanisme qui permet de tester si une action a réellement réussi, sans se fier uniquement à ce qui s'affiche à l'écran.

```
if useradd -m -g "$group" -s /bin/bash "$user" 2> /dev/null; then
    echo "Utilisateur créé"
else
    echo "Échec de la création"
fi
```

`2> /dev/null` redirige les messages d'erreur du système vers `/dev/null` (un fichier spécial qui efface tout ce qu'on y écrit), pour ne garder que le message contrôlé par le script.

5 Boucles et tableaux

```
services=("rh" "it" "finance" "direction")
for service in "${services[@]}; do
    echo "Traitement : $service"
done
```

Une boucle `for` parcourt chaque élément d'un tableau, en exécutant le même bloc d'instructions pour chacun — c'est le mécanisme qui remplace la répétition manuelle des 4 séquences de commandes.

6 Découper du texte sans commande externe

Pour associer chaque employé à son service (par exemple `rh_awa` au groupe `rh`), les noms ont été stockés sous la forme `"utilisateur:groupe"` dans un tableau, puis découpés directement en Bash :

```
entry="rh_awa:rh"
user="${entry%%:*}" # tout ce qui précède le premier ':' -> rh_awa
group="${entry##*:}" # tout ce qui suit le dernier ':' -> rh
```

Cette syntaxe évite de lancer un programme externe (comme `cut`) pour une opération aussi simple, ce qui est plus rapide sur un grand nombre d'itérations.

7 Les trois fonctions du script

Chaque fonction suit la même discipline : vérifier l'état existant avant d'agir, ne jamais recréer ce qui existe déjà. Cette propriété s'appelle l'**idempotence** — un script idempotent peut être relancé plusieurs fois de suite sans erreur ni doublon, produisant toujours le même résultat final.

`create_service` — créer un groupe s'il n'existe pas

```
create_service() {
    name_of_service="$1"
    similar_service=$(getent group "$name_of_service" | cut -d: -f 1)
    if [ "$name_of_service" = "$similar_service" ]; then
        echo "Le service $name_of_service existe déjà"
    else
        groupadd "$name_of_service"
        echo "Le service $name_of_service a été créé"
    fi
}
```

```
}
```

create_users — créer un utilisateur et l'attacher à son groupe

```
create_users() {
    user_name="$1"
    user_group="$2"
    user_exist=$(getent passwd "$user_name" | cut -d: -f 1)
    if [ "$user_exist" = "$user_name" ]; then
        echo "L'utilisateur $user_name existe déjà"
    else
        if useradd -m -g "$user_group" -s /bin/bash "$user_name" 2> /dev/null; then
            echo "L'utilisateur $user_name a été créé"
        else
            echo "Échec de la création de $user_name (code $?)"
        fi
    fi
}
```

setup_service_folders — dossier, permissions, SGID

```
setup_service_folders() {
    service_name="$1"
    path="/data/${service_name}_folders"
    if [ -d "$path" ]; then
        echo "Le dossier $service_name existe déjà"
    else
        mkdir -p "$path"
        echo "Le dossier $service_name a été créé"
    fi
    if ls -ld "$path" | grep -q "rwxrws"; then
        echo "Permissions déjà correctes"
    else
        chmod 770 "$path"
        chmod g+s "$path"
        echo "Permissions assignées"
    fi
    current_group=$(ls -ld "$path" | awk '{print $4}')
    if [ "$current_group" = "$service_name" ]; then
        echo "Groupe déjà correct"
    else
        chown :"$service_name" "$path"
        echo "Groupe assigné"
    fi
}
```

8 Assemblage final

```
services=("rh" "it" "finance" "direction")
employees=("rh_awa:rh" "rh_koffi:rh" "it_admin:it" "it_junior:it" \
    "finance_edwige:finance" "finance_bio:finance" "direction_ceo:direction")

for service in "${services[@]}; do
    create_service "$service"
    setup_service_folders "$service"
done
```

```
done

for entry in "${employees[@]}; do
    user="${entry%:*}"
    group="${entry##*:}"
    create_users "$user" "$group"
done
```

9 Preuve d'exécution : premier lancement (création)

Exécution sur un conteneur, sortie brute du terminal :

```
$ bash techcorp_setup.sh
rh creating...
The service rh was successfully created
Creating folder rh...
Assigning permissions to rh folder...
Assigning group rh to the folder...
finance creating...
The service finance was successfully created
Creating folder finance...
Assigning permissions to finance folder...
Assigning group finance to the folder...
direction creating...
The service direction was successfully created
Creating folder direction...
Assigning permissions to direction folder...
Assigning group direction to the folder...
user rh_awa is creating...
The user rh_awa was successfully created
user rh_koffi is creating...
The user rh_koffi was successfully created
user it_admin is creating...
The user it_admin was successfully created
user it_junior is creating...
The user it_junior was successfully created
user finance_edwige is creating...
The user finance_edwige was successfully created
user finance_bio is creating...
The user finance_bio was successfully created
user direction_ceo is creating...
The user direction_ceo was successfully created
```

10 Preuve d'exécution second lancement (idempotence)

Même commande, relancée immédiatement après, sans aucune modification du script ni de l'environnement :

```
$ bash techcorp_setup.sh
The service rh already exists on the server
The folder rh already exists
The folder for rh already has the right permissions
The folder for rh already has the right group
The service it already exists on the server
The folder it already exists
The folder for it already has the right permissions
The folder for it already has the right group
The service finance already exists on the server
The folder finance already exists
The folder for finance already has the right permissions
The folder for finance already has the right group
The service direction already exists on the server
The folder direction already exists
The folder for direction already has the right permissions
The folder for direction already has the right group
the user rh_awa already exists
the user rh_koffi already exists
the user it_admin already exists
the user it_junior already exists
```

```
the user finance_edwige already exists
the user finance_bio already exists
the user direction_ceo already exists
```

Aucune erreur, aucun doublon, aucune commande destructrice relancée inutilement : le script est confirmé idempotent.

11 Méthode de débogage suivie pendant la construction

Plusieurs erreurs réelles ont été rencontrées et corrigées pendant l'écriture de ce script — les lister ici a une valeur pédagogique directe, ce sont des pièges classiques du langage :

- `if "$variable"; then` — tente d'exécuter le contenu de la variable comme une commande, et non de tester sa valeur. Correction : tester directement le succès d'une commande (`if commande; then`) ou utiliser `[ ]` pour comparer du texte.
- `$(error: "$?")` — tentative d'utiliser une substitution de commande (`$()`) pour insérer du texte simple; `$()` exécute réellement ce qu'il contient. Correction : ne pas utiliser `$()` pour du texte fixe.
- `useradd` échouant silencieusement (groupe cible non créé au préalable) alors que le script affichait quand même « succès » — corrigé en testant le code de sortie réel de `useradd` avant d'afficher un message de confirmation.
- Comparaison de texte contre une ligne complète de `getent (nom:x:id:)` plutôt que contre le seul nom — corrigé avec `cut -d: -f 1` pour isoler le champ utile avant comparaison.

Conclusion

Le script final remplace une procédure manuelle de plus de 30 commandes, sujette à l'erreur humaine, par un outil unique, testé, idempotent, et documenté. Il peut être relancé sur un serveur neuf ou déjà partiellement configuré sans risque de duplication ni de corruption de l'état existant.